

Entrada y salida

Los dispositivos de entrada/salida permiten que la CPU se comunique con el mundo exterior. Ejemplos comunes de estos dispositivos incluyen teclados, pantallas, impresoras y discos.

La comunicación entre la CPU y un elemento externo es similar a la comunicación con la memoria principal, ya que en ambos casos se pueden leer y escribir datos. No obstante, el comportamiento de los dispositivos de entrada/salida es distinto. Por ejemplo, los datos no siempre están disponibles.

Debido a estas diferencias de comportamiento, los métodos de acceso utilizados para los dispositivos de entrada/salida también son distintos a los utilizados para acceder a la memoria.

Estructura de un sistema de entrada/salida

En teoría, los dispositivos de entrada/salida podrían comunicarse con la CPU a través de los buses del sistema. Sin embargo, debido a la gran heterogeneidad de estos dispositivos, sería muy costoso que la CPU los gestionara directamente. El sistema operativo tendría que conocer todos los detalles de cada dispositivo, y además, la CPU es mucho más rápida que la mayoría de los dispositivos de entrada/salida.

Por ello, los dispositivos están conectados a un componente hardware intermedio denominado **controlador de dispositivos**.

El controlador de dispositivo recibe comandos desde la CPU y se encarga de transmitirlos al dispositivo correspondiente. Por ejemplo, puede recibir la orden "escribir el bloque 2534 en el disco" y traducirla a las instrucciones físicas necesarias para realizar esa operación. De esta forma, la CPU queda liberada de tareas de muy bajo nivel, como especificar el lado, cilindro o sector del disco.

El controlador actúa, por tanto, como una interfaz entre la CPU y el dispositivo de entrada / salida. Puede gestionar uno o varios dispositivos del mismo tipo. Por ejemplo, un controlador IDE puede controlar dos discos duros, o bien un disco duro y una unidad DVD.

Comunicación entre controlador y CPU

Los controladores se comunican con la CPU mediante registros o puertos específicos, que suelen incluir:

- **Registros de control:** permite enviar comandos al dispositivo.

- **Registros de estado:** proporciona información sobre el estado del dispositivo o del propio controlador. Por ejemplo:
 - Si el comando anterior ya se ha completado.
 - Si se han producido errores en la última operación de entrada/salida.
- **Registros de datos:** pueden ser de entrada, de salida, o bidireccionales. Normalmente, son registros capaces de almacenar entre 1 y 8 bytes.

Algunos controladores integran chips FIFO que permiten almacenar temporalmente pequeñas cantidades de datos en ráfagas, hasta que la CPU o el dispositivo estén listos para recibirlos.

Funciones del controlador de dispositivo

El controlador es responsable de varias tareas fundamentales:

- **Coordinación de flujo de datos** entre la CPU y el dispositivo periférico.
- **Comunicación con el dispositivo de entrada/salida**, que incluye:
 - Envío de comandos.
 - Intercambio de datos.
 - Recepción de información sobre el estado del dispositivo.
- **Almacenamiento temporal de datos** (buffer), necesario debido a la gran diferencia de velocidad entre la CPU y el dispositivo. Por ejemplo, cuando la CPU envía datos más rápido de lo que el dispositivo puede procesarlos, el controlador actúa como intermediario, reteniéndolos y enviándolos de forma escalonada.
- **Detección de errores** durante la operación de entrada/salida

Estructura del software de entrada/salida

Abstracción y encapsulamiento

Un programa puede escribir en un disco duro sin conocer el tipo específico de disco gracias a la **abstracción**. Este principio permite eliminar las diferencias entre dispositivos al identificarlos dentro de unas pocas clases o tipos genéricos. Para cada clase se define una interfaz estándar con un conjunto fijo de funciones.

La **encapsulación** complementa este proceso: las diferencias reales entre dispositivos se ocultan dentro de los *device drivers*. Estos controladores se adaptan internamente a las particularidades de cada dispositivo, pero ofrecen hacia el exterior una interfaz estándar. Así, los programas pueden interactuar con los dispositivos sin conocer sus detalles internos.

Capas del software de E/S

El software de entrada/salida está estructurado en capas. Esta organización facilita tanto la incorporación de nuevos dispositivos como el desarrollo del sistema operativo.

- **Capa del controlador de dispositivos:** oculta las diferencias entre los distintos *device drivers* frente al núcleo del sistema. Gracias a esto, las llamadas al subsistema de E/S pueden tratar a los dispositivos como si perteneciesen a unas pocas clases genéricas.
- **Subsistema de E/S del sistema operativo:** se sitúa por encima de la capa de controladores, y permite que tanto los desarrolladores del sistema operativo como los fabricantes de hardware trabajen de forma más eficiente. Los fabricantes pueden o bien ajustar su hardware a interfaces existentes, o bien escribir el controlador de dispositivo adecuado para su integración.

Capas específicas del software de E/S

El software de entrada/salida se divide en las siguientes capas funcionales:

- **Software a nivel de usuario:** proporciona la interfaz entre el usuario y el sistema. Incluye tareas como el formateo de datos.
- **Software independiente del dispositivo:** se encarga de tareas como la asignación de espacio, el control de privilegios o el uso de caché, en caso de existir. No depende del dispositivo físico concreto.
- **Controlador de dispositivo:** es la única capa que se comunica directamente con el controlador del dispositivo. Toda la lógica específica del dispositivo está contenida en esta capa. Para que un sistema operativo pueda utilizar un nuevo dispositivo, basta con incorporar su correspondiente *driver*.
- **Gestión de interrupciones:** se encarga de gestionar las interrupciones que provienen de los dispositivos, despertando procesos que esperaban una operación de E/S o liberando recursos.

Tipos de entrada/salida

Según la forma en que se realiza la comunicación entre la CPU y los dispositivos, y según la percepción que tiene el proceso respecto al desarrollo de la operación de entrada/salida, podemos distinguir varios tipos de E/S.

Según el método de comunicación entre la CPU y los dispositivos

Entrada/Salida mapeada en memoria

En este modelo, los dispositivos aparecen dentro del espacio de direcciones. Es decir, la memoria y los dispositivos comparten el mismo espacio de direcciones.

Esto no suele suponer un problema, ya que el espacio necesario para E/S suele ser mucho

menor que el espacio total de direcciones disponible. Para acceder a los registros del controlador del dispositivo se utilizan instrucciones del tipo MOV, como en cualquier operación normal de lectura o escritura en memoria.

Entrada/salida con espacio de puertos separado

En este caso, se dispone de un conjunto de direcciones explícitas dedicadas exclusivamente a E/S. A cada registro de control de un dispositivo se le asigna un número de puerto E/S, y el conjunto de todos ellos forma lo que se conoce como *espacio de entrada/salida*.

Normalmente, se emplean instrucciones especiales para acceder a este espacio, como IN o OUT en la arquitectura Intel. En otros sistemas, como PowerPC, un registro de control puede activar el uso de este espacio.

Según la percepción del proceso sobre la operación de E/S

Entrada/Salida Sincrónica

En este caso, el proceso percibe que debe esperar hasta que la operación de E/S se complete. Desde su punto de vista, no puede continuar su ejecución hasta que haya finalizado la entrada o salida.

Entrada/Salida Asíncrona

Aquí, el proceso percibe que no necesita esperar a que la operación de E/S termine. Puede seguir ejecutándose y, en algún momento, será notificado de que la operación ha finalizado.

Dado que la CPU es mucho más rápida que los dispositivos de E/S, el sistema operativo cede el uso de la CPU a otra tarea mientras la operación de entrada/salida se lleva a cabo. El proceso que inició la operación pasa a estado de *espera*. Aunque el proceso "cree" que la operación es sincrónica, en realidad el sistema la está manejando de forma asíncrona. Cuando la E/S concluye, el proceso pasa para de la cola de I/O a la cola de listos.

Además, muchos sistemas operativos permiten realizar operaciones de entrada/salida verdaderamente asíncronas. En este caso, el proceso inicia la operación y continúa su ejecución sin interrupciones. Más adelante, será notificado por el sistema operativo cuando la operación haya concluido.

Esta notificación puede realizarse, por ejemplo:

- Mediante una interrupción.
- Modificando el valor de una variable en el espacio de usuario del proceso.

Métodos para realizar operaciones Entrada/Salida

Existen distintos métodos para llevar a cabo operaciones de entrada/salida entre la CPU y los dispositivos. A continuación se describen los principales enfoques utilizados: mediante sondeo (polling), mediante interrupciones, y mediante acceso directo a memoria.

Entrada/Salida con sondeo

Es el método más simple para realizar una operación de entrada/salida.

Para lograr la sincronización entre el dispositivo y la CPU, esta última debe consultar repetidamente al dispositivo para comprobar si tienen un nuevo dato disponible para ser leído o si está listo para recibir un nuevo dato.

Este enfoque genera lo que se conoce como espera activa, en la que se desperdicia tiempo de CPU cada vez que se realiza una consulta al dispositivo. Cuanto más frecuente sea esta consulta, más tiempo se pierde.

El sondeo resulta lento y, además, puede dar lugar a la pérdida de datos si el dispositivo no es consultado con la suficiente frecuencia.

Entrada/Salida controlada por interrupciones

En este método, es el propio dispositivo el que solicita la atención de la CPU mediante una interrupción. Cuando se detecta una interrupción, el sistema operativo:

1. Detiene el proceso que estaba en ejecución y guarda su estado.
2. Transfiere el control al *interrupt handler*
3. Ejecuta dicho procedimiento.
4. Una vez finalizado, continúa la ejecución del proceso detenido desde el punto en el que se encontraba.

Estos procedimientos de servicio están ubicados en posiciones de memoria indicadas por los vectores de interrupción. Durante el proceso de inicialización, el sistema operativo se encarga de comprobar qué dispositivos están conectados e instala los correspondientes *drivers* en el vector de interrupciones.

El vector de interrupciones se encarga de dirigir cada interrupción al manejador adecuado.

Una vez finalizado el servicio de interrupción, normalmente se invoca al planificador de tareas, y el dispositivo, por ejemplo una impresora, vuelve a generar una interrupción cuando está listo para continuar con la transferencia del siguiente dato.

Entrada/Salida mediante acceso directo a memoria

Tanto en la entrada/salida con sondeo como con interrupciones, la CPU es quien dirige la transferencia de datos cuando estos están listos, bien sea consultando el registro de estado o recibiendo una interrupción del dispositivo.

Esto presenta algunos inconvenientes:

- La velocidad de transferencia está limitada por la velocidad de la CPU al mover los datos entre los registros/memoria y el controlador o dispositivo.
- Mientras se realiza la transferencia, la CPU debe encargarse de dirigir dicha operación, impidiéndole atender otras tareas.

Aunque estos problemas son menores en dispositivos lentos que transfieren poca cantidad de datos, se vuelven críticos en dispositivos que manejan grandes volúmenes de información, como los discos duros.

Para resolverlo, se utiliza un controlador de DMA, que libera a la CPU de la gestión directa de la transferencia de datos. En este caso, la CPU sólo interviene al final del proceso, cuando recibe una interrupción que le notifica la finalización de la transferencia.

El controlador DMA se encarga por completo de transferir los datos al dispositivo. Genera todas las señales de dirección y control del bus necesarias. La CPU debe proporcionar al DMA tres elementos:

- El tipo de operación,
- La dirección de memoria involucrada en la transferencia,
- El número de bytes a transferir.

Cuando la transferencia concluye, el DMA genera una interrupción para avisar a la CPU. Como tanto el DMA como la CPU deben utilizar el mismo bus, existen distintos métodos para compartirlo:

Modos de compartir el bus

1. DMA en modo ráfaga

Una vez que el DMA toma el control del bus, transfiere un bloque completo de datos. La CPU debe esperar hasta que finalice toda la transferencia.

Durante este tiempo, la CPU puede seguir ejecutando instrucciones y accediendo a datos desde la caché. Este es el método más rápido.

Se utiliza principalmente en dispositivos de almacenamiento secundario como discos, ya que al ser costoso obtener acceso al bus, se aprovecha para transferir varios datos de una vez.

2. DMA con robo de ciclos

El DMA transfiere una sola palabra cada vez que toma el control del bus, y luego cede el control nuevamente a la CPU.

La transferencia de datos se realiza mediante una secuencia de ciclos DMA intercalados con ciclos de CPU.

3. Bus transparente

El DMA sólo usa el bus cuando la CPU no lo está utilizando, por ejemplo, mientras la CPU está decodificando una instrucción o utilizando la ALU.

Durante la transferencia, la CPU puede seguir accediendo a datos e instrucciones que se encuentren en la caché.

Estructura de discos

Un disco tradicional está formado por un conjunto de platos que giran de manera conjunta. Cada una de las superficies de los platos se denominan cara o lado, y cada una de ellas contiene una serie de pistas circulares concéntricas.

El conjunto de pistas que se sitúan en la misma posición de los distintos lados conforma lo que se llama un cilindro. Cada cilindro, a su vez, está dividido en sectores, que constituyen la unidad básica de entrada/salida en un disco duro. Típicamente, cada sector tiene una capacidad de 512 bytes.

Para referirse de forma precisa a un sector, se utilizan tres coordenadas: el cilindro, el lado y el número de sector. Por convención, la numeración de cilindros y lados comienza desde 0, mientras que los sectores suelen numerarse a partir de 1.

El sistema operativo no accede directamente a las características del disco. En su lugar, lo visualiza como una secuencia de bloques lógicos. Por ejemplo, un disquete de 1,4 MB tiene 80 cilindros, 2 lados y 18 sectores por pista. Si este disco se formatea con bloques de 4 KB, el sistema operativo lo tratará como una secuencia de 360 bloques lógicos, cada uno de ellos formado por 8 sectores físicos.

La asignación del espacio se realiza a nivel de bloques lógicos, no de sectores físicos.

Particiones de disco

Los discos de gran tamaño pueden dividirse en una o más áreas, conocidas como particiones. Cada partición puede utilizarse como un sistema de ficheros independiente.

De forma tradicional, el primer sector del disco contiene una tabla que describe las distintas particiones del disco. El formato de esta tabla varía según el sistema operativo:

- En sistemas Windows y Linux, se utiliza una tabla de particiones de tipo MBR, que permite definir hasta 4 entradas.
- En sistemas con particiones de tipo GPT, se pueden definir hasta 128 entradas, aunque la tabla de particiones ocupa más de 512 bytes.

- En sistemas BSD se emplea una estructura denominada *disklabel*, que puede contener 8 o 16 entradas.

Planificación de discos

Cuando en un sistema se generan múltiples particiones de entrada/salida dirigidas al disco, estas pueden organizarse mediante algoritmos de planificación. La planificación de discos es responsabilidad del sistema operativo, cuyo objetivo es usar el hardware de forma eficiente. En el caso de los discos, esto implica minimizar el tiempo de acceso.

El tiempo de acceso a un bloque de disco tiene dos componentes principales:

- **Tiempo de búsqueda:** es el tiempo necesario para mover las cabezas del disco hasta el cilindro que contiene el bloque deseado.
- **Latencia rotacional:** es el tiempo que se espera a que el disco gire y posicione el sector deseado bajo la cabeza de lectura/escritura.

Otros tiempos, como el de transferencia de bloque, suelen considerarse constantes. Dado que el tiempo de búsqueda suele ser el factor dominante, la planificación de discos se enfoca habitualmente en reducirlo.

Aunque también existen técnicas para reducir la latencia rotacional, en sistemas modernos el controlador suele leer pistas completas y luego transfiere únicamente los sectores requeridos.

Algoritmos de planificación de disco

Existen varios algoritmos para decidir el orden en que se atienden las peticiones de disco:

- **First Come First Served:** Las peticiones se atienden en orden de llegada.
- **Shortest Seek Time First:** Se atienden primero las peticiones que requieran menor desplazamiento de la cabeza desde su posición actual. Aunque reduce el tiempo medio de búsqueda, puede provocar inanición de algunas peticiones alejadas.
- **SCAN:** La cabeza del disco se desplaza en una dirección, atendiendo todas las peticiones en su camino. Al llegar al extremo, invierte su dirección y continúa sirviendo las peticiones pendientes en el nuevo sentido.
- **C-SCAN:** Similar a SCAN, pero cuando la cabeza alcanza el final del disco, vuelve directamente al inicio sin atender ninguna petición en el trayecto de regreso. Esto proporciona un tiempo de espera más uniforme para cada cilindro.
- **SCAN /C-SCAN con N pasos:** Son variantes de los anteriores, pero en cada recorrido la cabeza atiende como máximo N peticiones.
- **C-LOOK:** Variante de C-SCAN. En lugar de llegar hasta el último cilindro, la cabeza se detiene en el último cilindro que tiene una petición pendiente. De igual forma, al cambiar de

dirección no necesariamente vuelve al cilindro 0, sino al primero con petición pendiente.

Otros algoritmos de planificación

- **Complete Fair Queuing:** Este algoritmo asigna distintas colas a las peticiones de disco provenientes de distintos procesos. A cada cola se le otorga un quantum de tiempo, cuya duración y número de peticiones que pueden procesarse dependen de la prioridad de entrada/salida (i/o) del proceso. En Linux, el comando `ionice` permite modificar dicha prioridad.
- Existen otros algoritmos en Linux, como *noop*, *deadline* o *anticipatory scheduling*, que ofrecen distintas estrategias según las necesidades del sistema.

Entrada/Salida en UNIX

Dispositivos de entrada/salida en UNIX

En Unix, los dispositivos aparecen representados como archivos, habitualmente ubicados en el directorio `/dev`.

En algunos sistemas, como Solaris, estos archivos pueden ser enlaces simbólicos que apuntan al archivo real del dispositivo.

Cada dispositivo se asocia a un inodo. Este inodo indica si se trata de un dispositivo de bloque o de carácter, y contiene dos números: el número mayor (major) y el número menor (minor). El número mayor identifica el manejador de dispositivo (driver) que se debe utilizar para acceder al dispositivo, mientras que el número menor especifica cuál unidad concreta, de las gestionadas por ese manejador, se está utilizando.

Podemos acceder a los dispositivos mediante las mismas llamadas del sistema que se usan para acceder a archivos (`open`, `read`, `write`), siempre y cuando el proceso tenga los privilegios adecuados. Para acceder a características o funcionalidades adicionales del dispositivo, se utiliza la llamada `ioctl`.

Llamadas de E/S en UNIX

En UNIX se usa E/S no formateada: se leen o escriben secuencias de bytes. Las llamadas de E/S reciben un descriptor de archivo, una dirección de memoria y el número de bytes a transferir.

```
int open(const char *path, int flags, mode_t mode);
```

Recibe la ruta del archivo (`path`), un modo de apertura, como `O_RDONLY`, `O_WRONLY`, `O_RDWR`, y en caso de creación, los permisos. Devuelve un descriptor de archivo, que se usará en las siguientes llamadas a `read` o `write`.

```
int close(int fd);
```

Cierra el descriptor de archivo `fd`, previamente obtenido mediante `open`.

```
ssize_t read(int fd, void *buf, size_t count);
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

Estas llamadas reciben un descriptor de archivo, una dirección de memoria y el número de bytes a transferir. Devuelven el número de bytes efectivamente leídos o escritos. La operación se realiza desde la posición actual del archivo, que se actualiza tras cada llamada.

```
off_t lseek(int fd, off_t offset, int whence);
```

Permite modificar la posición actual dentro del archivo. El desplazamiento se interpreta según el valor de `whence`:

- `SEEK_SET`: desde el principio del archivo.
- `SEEK_CUR`: desde la posición actual.
- `SEEK_END`: desde el final del archivo.

```
ssize_t pread(int fd, void *buf, size_t count, off_t offset);
```

```
ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);
```

Funcionan como `read` y `write`, pero con un parámetro adicional que indica el desplazamiento desde el inicio del archivo. Estas llamadas **no** modifican la posición actual.

```
ssize_t readv(int fd, const struct iovec *iov, int iovcnt);
```

```
ssize_t writev(int fd, const struct iovec *iov, int iovcnt);
```

```
ssize_t preadv(int fd, const struct iovec *iov, int iovcnt, off_t offset);
```

```
ssize_t pwritev(int fd, const struct iovec *iov, int iovcnt, off_t offset);
```

Estas llamadas permiten realizar operaciones de E/S sobre varias regiones de memoria no contiguas. Cada `struct iovec` describe un búfer: su dirección (`iov_base`) y su tamaño en bytes (`iov_len`).

Entrada/Salida asíncrona

En Unix, los procesos pueden realizar E/S de manera asíncrona, es decir, iniciar una operación de entrada/salida y recibir una notificación cuando esta se completa.

Las funciones que permiten este tipo de operación pertenecen al estándar POSIX de tiempo real y deben entrelazarse con `-lrt`.

Funciones principales

- `int aio_read(struct aiocb *aiocbp);`
- `int aio_write(struct aiocb *aiocbp);`
- `ssize_t aio_return(struct aiocb *aiocbp);`
- `int aio_error(struct aiocb *aiocbp);`
- `int aio_cancel(struct aiocb *aiocbp);`

```
struct aiocb {  
    int aio_fildes;           // Descriptor de archivo
```

```

off_t aio_offset;           // Desplazamiento dentro del archivo
volatile void *aio_buff;    // Dirección del búfer
size_t aio_nbytes;         // Número de bytes a transferir
int aio_reqprio;           // Prioridad de la petición
struct sigevent aio_sigevent; // Método de notificación
int aio_lio_opcode;        // Operación (sólo para lio_listio)
...
}

```

Notificación

El campo `aio_sigevent` define cómo debe notificarse al proceso cuando la operación finaliza. El tipo de notificación puede tomar valores como:

- `SIGEV_NONE`
- `SIGEV_SIGNAL`
- `SIGEV_THREAD`

Otros campos relevantes son `sigev_signo`, `sigev_value`, `sigev_notify_function`, `sigev_notify_attributes` y `sigev_notify_thread_id`.

Redirección de entrada, salida y error

Los descriptors de archivo 0, 1 y 2 corresponden respectivamente a la entrada estándar (`STDIN_FILENO`), salida estándar (`STDOUT_FILENO`) y error estándar (`STDERR_FILENO`) de un proceso. Estos flujos pueden redirigirse a archivos usando la shell. Por ejemplo, en Bash se utilizan los símbolos:

- `<` para redirigir la entrada.
- `>` para redirigir la salida.
- `&>` para redirigir el error.

Redirección mediante llamadas del sistema

Para hacer redirección de flujos desde código, pueden utilizarse las llamadas `dup`, `dup2` y `fcntl` con opción `F_DUPFD`.

Uso de `dup`: `int dup(int fd);`

Esta función duplica un descriptor de archivo, asignando el número más bajo disponible. El descriptor nuevo y el original se refieren al mismo archivo abierto, compartiendo sus indicadores y la posición actual (`cur_pos`). Por tanto, una llamada a `lseek` en uno de ellos afectará también al otro.

1. [Introducción a sistemas operativos](#)

2. [Ficheros](#)

3. [Memoria](#)

4. [Procesos](#)

5. [Entrada Salida](#)